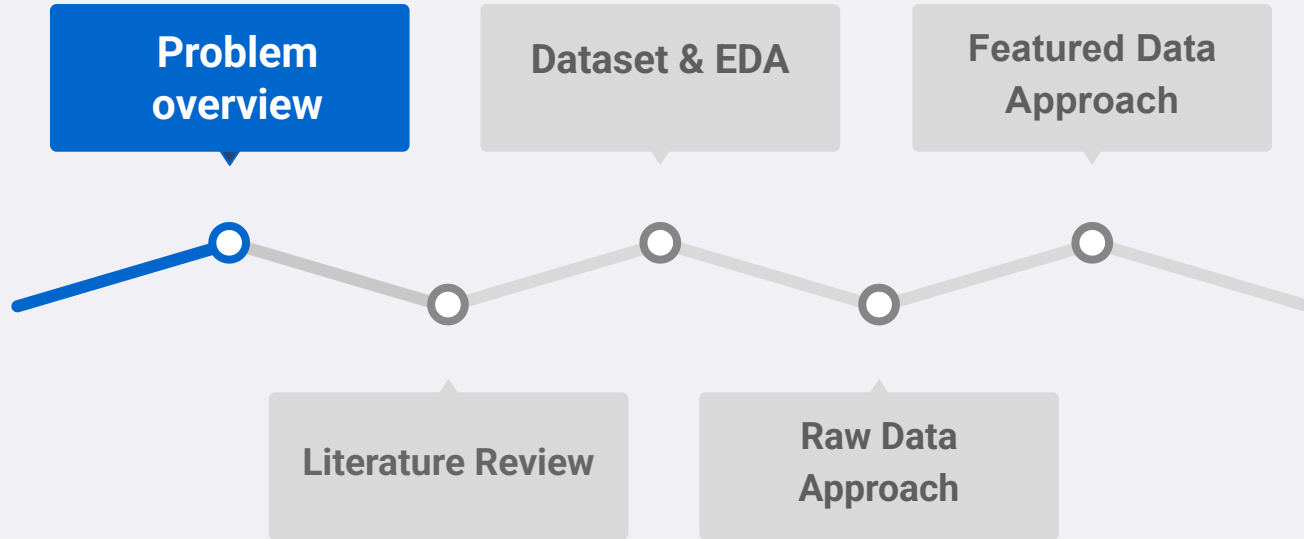


Detecting Sleep Apnea from Raw Physiological Signals

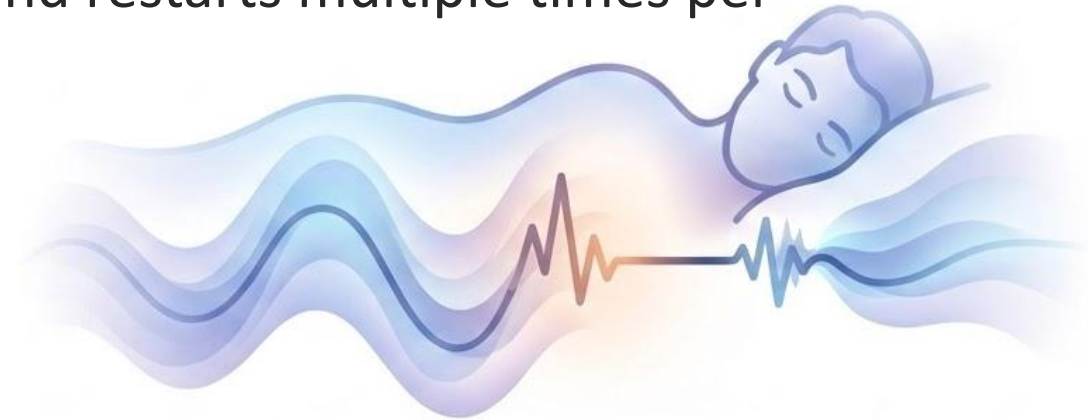
Giulia Sepeda Martins Silveira
Vikram Vupparapalli



Sleep Apnea

Definition

- Serious sleep condition
- Breathing stops and restarts multiple times per night



Sleep Apnea

Prevalence

20%

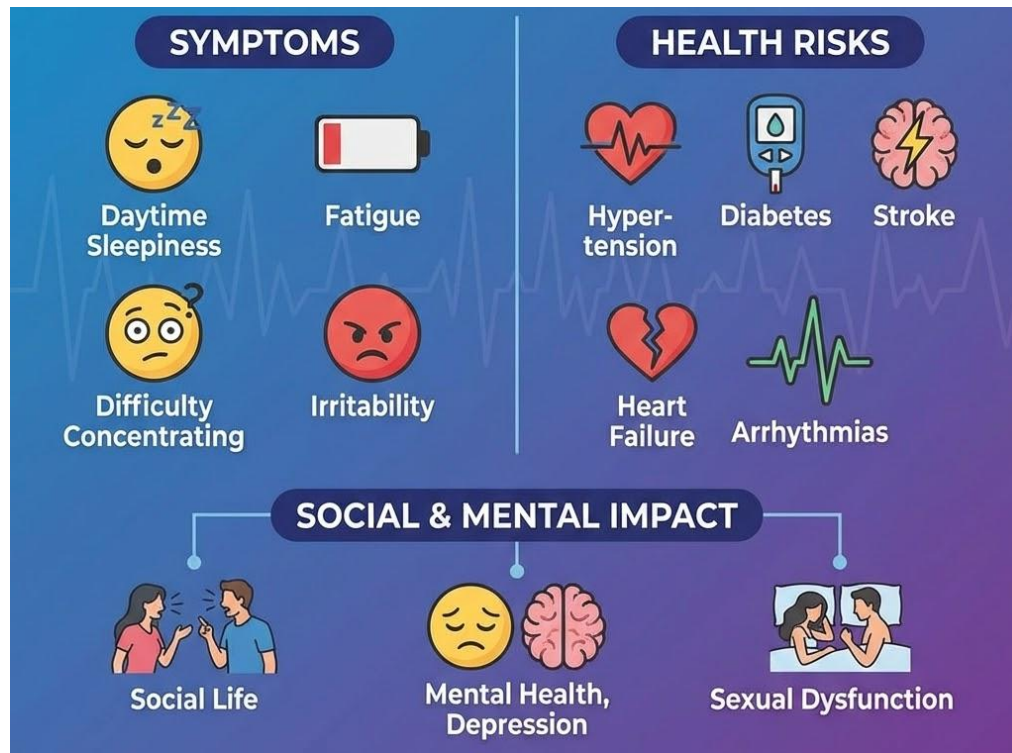
Globally

80%

Undiagnosed

Sleep Apnea

Consequences



Sleep Apnea

Diagnosis



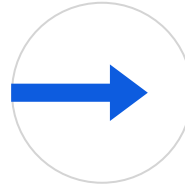
Multi-channel physiological signals recorded overnight

The Challenge

Goal

Manual PSG scoring

- Time-consuming
- Subjective



Automatic detection

Time-series binary classification

The Challenge

Goal

Manual PSG scoring



Time-consuming



Subjective

Automatic detection



Time-series binary
classification

The Dataset

Only 22
labeled

44 patients

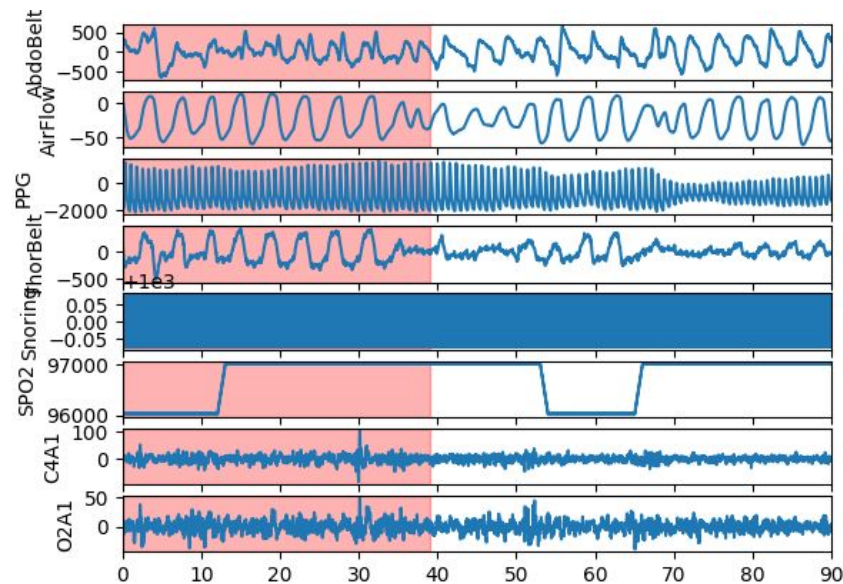
8 channels

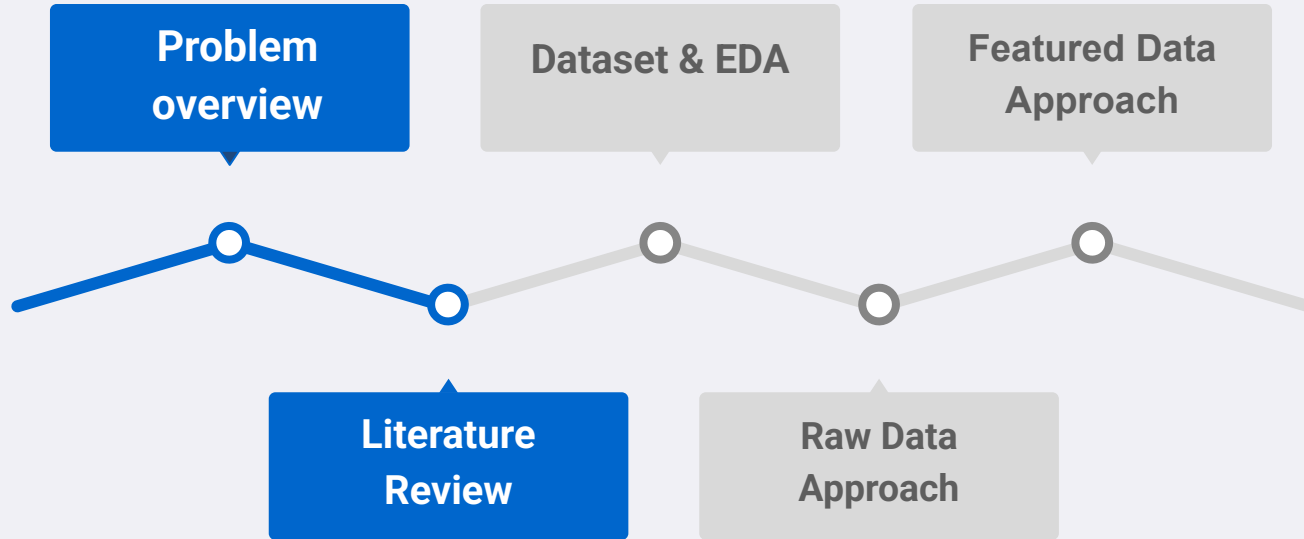
100 Hz

PSG

90s windows
no overlap

Expert-
annotated





Literature Review

Large Datasets

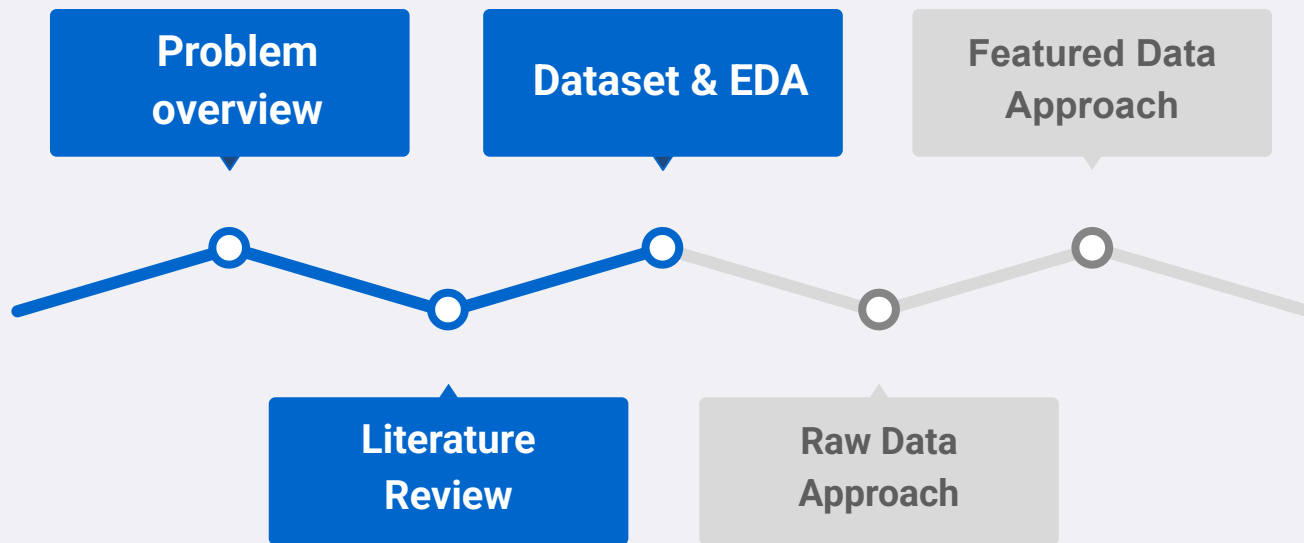
- Large Cohorts
- training on Raw data
- CNN/CNN-LSTMs

Smaller Datasets

Physiological features
+ Tree-based models

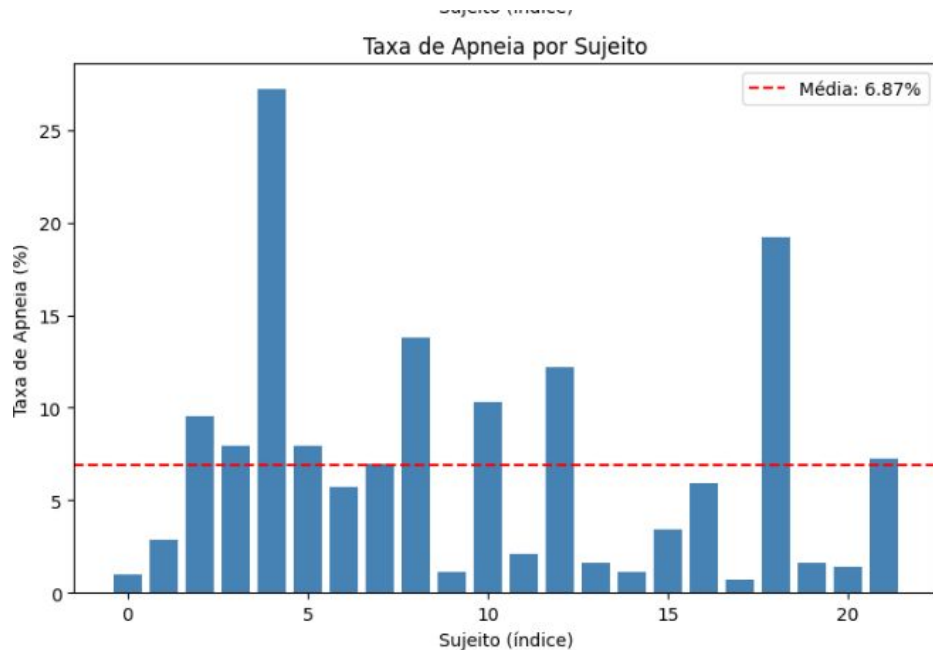
Performance

Event-level metrics -> better clinical relevance
Small, heterogeneous datasets -> drop performance

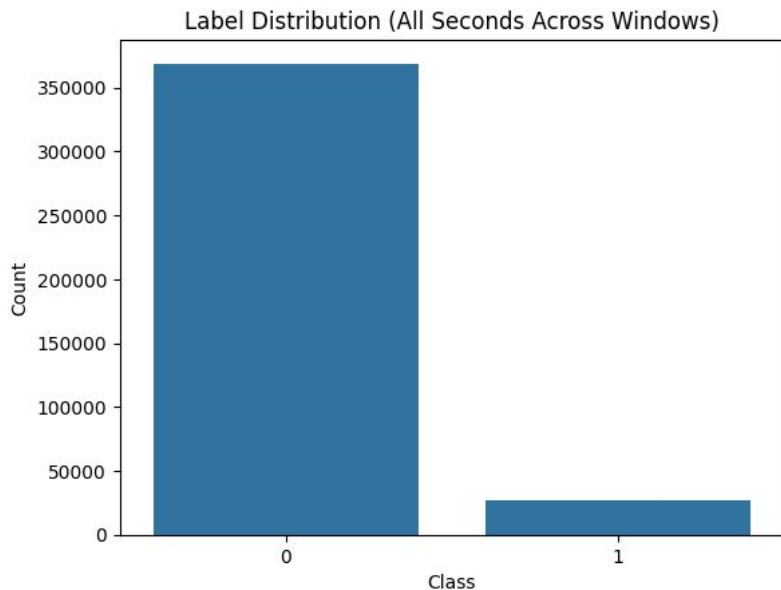


Dataset Characteristics – Training Data

- **Only 22** subjects available
- Strong **inter-subject heterogeneity**
- **Subject-level splits**
- K fold
- **Overlapping windows**
 - temporal samples
 - give context



Dataset Characteristics – Rare event



Event appears **only 6.9%** of the events

Loss Function (Weighted BCE)

```
positive_ratio = float(np.mean(y))
neg_ratio = 1.0 - positive_ratio
pos_weight = neg_ratio / (positive_ratio + 1e-12)
print("Positive ratio:", positive_ratio, "| pos_weight:", pos_weight)

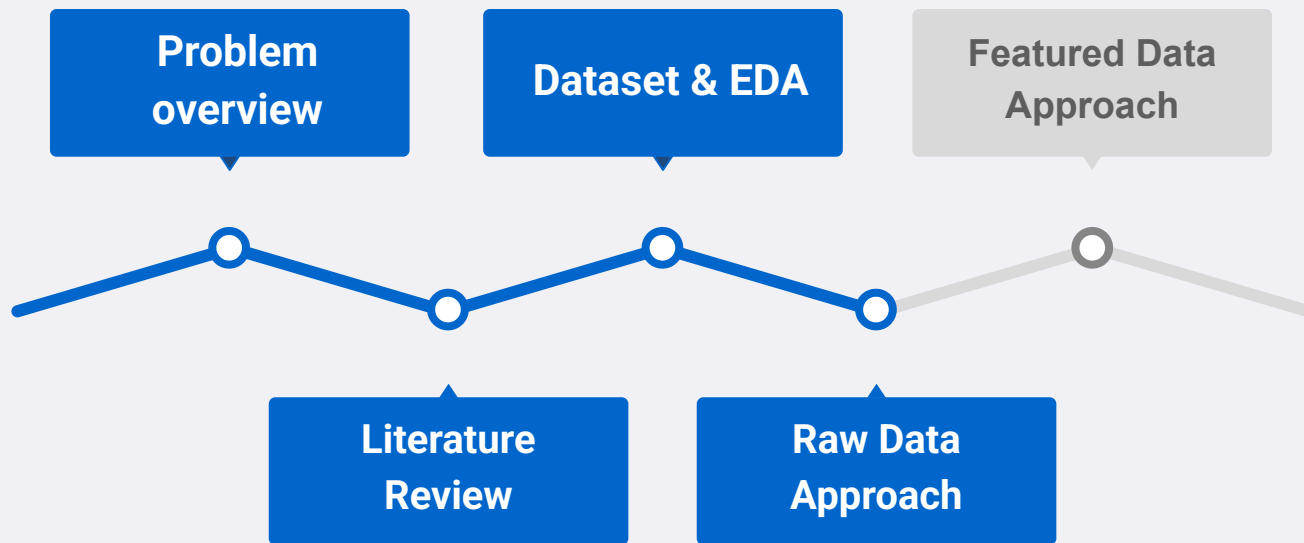
def weighted_bce(pos_weight):
    def loss(y_true, y_pred):
        bce = K.binary_crossentropy(y_true, y_pred)
        w = 1.0 + (pos_weight - 1.0) * y_true
        return K.mean(bce * w)
    return loss
```

✓ 0.0s

Positive ratio: 0.0687222222222223 | pos_weight: 13.551333872074434

Dataset Characteristics – Preprocessing

- Two normalization strategies tested:
 - **Window-wise** z-score
 - **Subject-wise** z-score -> **preserves physiological baseline**
- Final models use subject-wise normalization



Baseline Model

Raw Data - Original 90s-window

- Input: 9000, 8
- 2-layer 1D ConvPool + 1-layer Conv - challenge reference architecture
- Weighted_BCE

```
def build_baseline_cnn():  
    model = models.Sequential([  
        layers.Input(shape=(N_SAMPLES, N_CHANNELS)),  
  
        layers.Conv1D(32, kernel_size=25, padding='same',  
            activation='relu'),  
        layers.MaxPooling1D(pool_size=10), # 9000 → 900  
  
        layers.Conv1D(64, kernel_size=25, padding='same',  
            activation='relu'),  
        layers.MaxPooling1D(pool_size=10), # 900 → 90  
  
        layers.Conv1D(1, kernel_size=3, padding='same',  
            activation='sigmoid'),  
        layers.Lambda(lambda t: tf.squeeze(t, axis=-1)) #  
    ])  
    return model
```

val_loss: 1.4340

Baseline Model

insights

- **Change on learning rate** = big change 1e-6 to **1e-4 + callback**
- Windows z-score giving false info?
- Train threshold $\neq$ test threshold
- Right metric
- “Broken” event

lr 1e-6 threshold 0.495	0.0011350737797956867
lr 1e-6 threshold 0.5	0.0

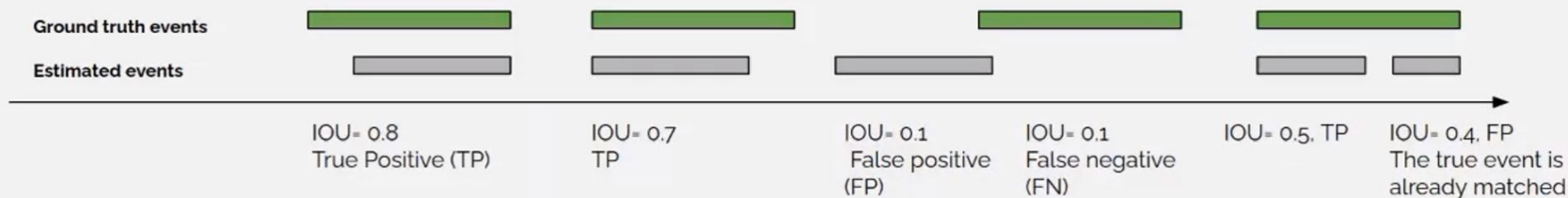
3l cnn opt threshold 0.54	0.040565059616381544
3l cnn opt threshold 0.55	0.017142237170445915

	precision	recall	f1-score
no_apnea	0.82	0.93	0.87
apnea	0.58	0.34	0.43

Best threshold: 0.5
Best window-level F1: 0.42745

Evaluation Metric - Event F1

1. Events start and end are computed from the binary mask
2. Ground truth and target events are matched if their Intersection Over Union (IoU) is greater than 0.3



3. The F1-score over matched events is the risk function:

$$\mathbf{R}_t = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{TP}{TP + 0.5(FP + FN)} = 0.666$$

CNN Model

Raw Data - 300s-windows

- Night normalization
 - different patient = different baseline
 - Pre and post event context
- LR **callback**
- Right **metric**
- Better post processing with **clinical judgment**

```
BEST postproc Event-F1: 0.2581344902381185  
params (t, min_len, gap_fill): (0.56, 12, 2)  
Percentage of 1s after post-processing: 8.03011
```

```
submission shape: (4400, 91)  
IDs unique: True  
overall %ones: 8.770454545454545
```

```
t = 0.535  
min_len = 6  
gap_fill = 3
```

CNN Model

Raw Data - 300s-windows

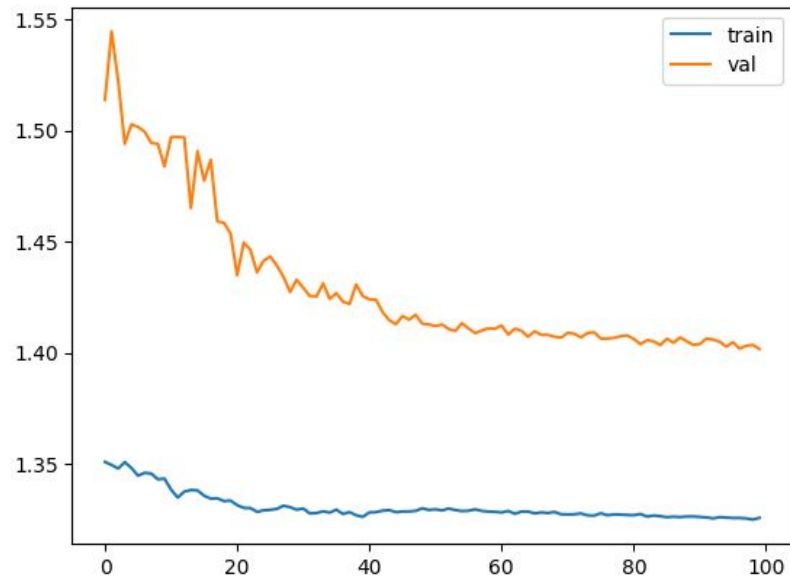
- Input: 30000, 8
- Same 2-layer 1D ConvPool + 1-layer Conv
- Longer training ~2h40

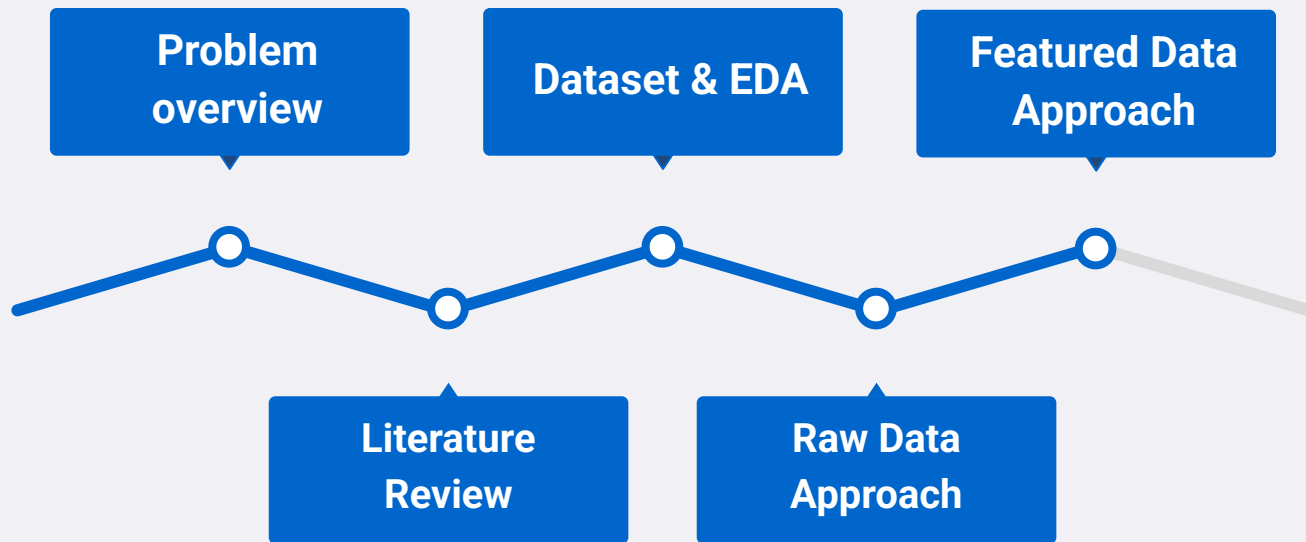
t = 0.535 min_len = 6 gap_fill = 3

0.0739404869251571

t = 0.52 min_len = 12 gap_fill = 3

0.072709646146388





Feature Engineering

- After Subject-wise normalization
- **18 variables** created using:
 - Amplitude-based statistics
 - Rolling window aggregations
 - Relative change metrics
 - Temporal summaries
 - Trend & dynamics
 - Interaction terms
- Most relevant features **capture classic apnea patterns**

	feature	importance
6	airflow_baseline_local	5087.4
13	spo_min_c30	4965.2
14	spo_range_c30	3678.0
7	airflow_drop_pct_c30	3365.2
5	spo_proxy_mean_1s	2557.0
11	thor_abd_corr_c30	2484.0
16	airflow_drop_x_spo_range	2275.6
17	airflow_lowdur_x_spo_rng	1883.8

LightGBM

fold	AUC_1Hz	AP_1Hz	best_thr	event_F1	n_val_subjects	pos_rate_val
1	0.675968	0.143455	0.45	0.061856	5	0.076822
2	0.672660	0.087003	0.20	0.047138	5	0.048111
3	0.674344	0.076999	0.35	0.039643	4	0.041750
4	0.668861	0.130391	0.35	0.055633	4	0.073597
5	0.630571	0.179171	0.25	0.077910	4	0.106458

```
smooth_win: 1
thr_on: 0.15
thr_off: 0.1
min_len: 5
max_gap: 3
event_F1: 0.085148
```

47	Jan. 21, 2026, 7:21 p.m.	giuuusepeda & Vikram	0.0854
----	--------------------------	----------------------	--------

Model Comparison Results

	Input	Norm.	Architecture	Threshold	F1 (val)	F1 (test)
Baseline original windows	Raw data original windows	Window wise	2 Convpool + 1 Conv	t = 0.58 min_len=10 gap_fill = 0	0.2513	0.0406
2nd model windows resplit	Raw 300s windows 80% overlap	Subject wise	2 Convpool + 1 Conv	t = 0.535 min_len = 6 gap_fill = 3	0.2470	0.0739
3rd model featured	Featured	Pre feature Subject wise	LightGBM w 5 K-Fold Ensemble	t = 0.11 Min_len = 1 gap_fill = 3	0.0851	0.0854

Split 70/30 always per subject and Weighted BCG as loss function

Challenges & insights

Limited training data	Sensitive data → limited availability
Class imbalance	Weighted binary cross-entropy
Inter-subject heterogeneity	Subject-wise (Z-fold) splitting
Threshold tuning	Clinical expertise + positive overall %
Computational limitation for other types of NN	Future work: cloud & larger models

Results

- Compared two modeling paradigms
 - Tree-based models vs deep learning
- Consistent with prior work
 - Feature engineering + tree-based models performed better with limited data
- Competitive challenge performance
 - Ranked 47th in the challenge (opened in 2021)

Detecting Sleep Apnea from Raw Physiological Signals

Giulia Sepeda Martins Silveira
Vikram Vupparapalli

Thank You! Questions?

Dataset Characteristics – Unprocessed

- X (22, 8, 9000)
 - H5
 - 8 channel
 - 100 hz
 - 44 subjects
- y (4400, 91):
 - csv
 - 200 90s-windows
 - 22 subjects

	Channel	Mean	Std	Min	Max
0	Abdominal belt	5.888035	480.813087	-1145.483551	1181.553592
1	Airflow	-28.403486	57.311850	-455.254483	351.567086
2	PPG	-857.776287	1513.287953	-4995.011702	4348.314102
3	Thoracic belt	-9.012861	432.779134	-1187.903087	1183.393149
4	Snoring	693.812603	463.935759	-1171.444646	1224.009839
5	SpO2	91942.331172	16592.621038	6.680096	105313.695534
6	EEG C4-A1	0.000448	21.452712	-2521.738076	2555.461083
7	EEG O2-A1	-0.000134	20.702043	-2667.758233	2660.121977

	ID	y_0	y_1	y_2	y_3	y_4	y_5	y_6	y_7	y_8	...	y_80	y_81	y_82	y_83
0	0	0	0	0	0	0	0	0	0	0	...	0	0	0	0
1	1	0	0	0	0	0	0	0	0	0	...	0	0	0	0
2	2	0	0	0	0	0	0	0	0	0	...	0	0	0	0
3	3	0	0	0	0	0	0	0	0	0	...	0	0	0	0
4	4	0	0	0	0	0	0	0	0	0	...	0	0	0	0

CNN Callbacks

```
callbacks = [  
    tf.keras.callbacks.EarlyStopping(  
        monitor="val_loss",  
        patience=10,  
        restore_best_weights=True,  
        verbose=1  
    ),  
    tf.keras.callbacks.ModelCheckpoint(  
        filepath=ckpt_best,  
        save_best_only=True,  
        monitor="val_loss",  
        verbose=1  
    ),  
    tf.keras.callbacks.ReduceLROnPlateau(  
        monitor="val_loss",  
        factor=0.5,  
        patience=3,  
        min_lr=1e-6,  
        verbose=1  
    )  
]
```

LightGBM Parameters

n_estimators: 600

learning_rate: 0.03

num_leaves: 63

subsample: 0.8

colsample_bytree: 0.8

min_child_samples: 200

reg_lambda: 1.0

class_weight: 'balanced'

random_state: 42

n_jobs: -1

thr_global: 0.30000000000000004

pp_min_len: 10

pp_merge_gap: 5

iou_thr: 0.1